RAG-based Customer Support Assistant

HIGH-LEVEL DESIGN (HLD)

1. System Overview

- ## Problem Definition

Traditional customer support systems face several limitations:

Existing Challenges

- **Generic Responses**
  Rule-based bots provide predefined answers without understanding context.
- **No Document Grounding**
  Systems do not utilize internal documents like policies, manuals, or FAQs effectively.
- **Poor Domain Adaptability**
  Cannot handle organization-specific queries (e.g., HR policies, refund rules).
- **High Human Dependency**
  Complex queries require manual intervention, increasing operational cost.

Proposed Solution: RAG-Based Support System

A Retrieval-Augmented Generation (RAG) system that:

- Accepts structured/unstructured documents (PDFs, FAQs, policies)
- Converts them into searchable embeddings
- Retrieves relevant context dynamically
- Uses an LLM to generate accurate, context-aware answers
- Integrates Human-in-the-Loop (HITL) for edge cases

Scope of the System

In Scope:

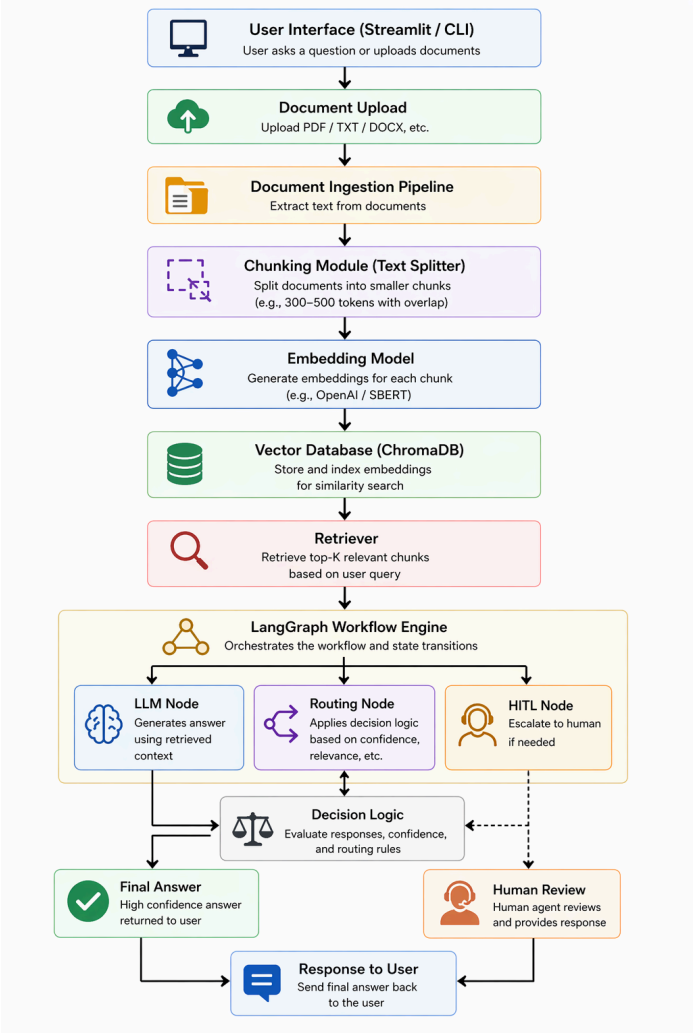
- PDF upload and processing
- Context-aware query answering
- Multi-session support
- HITL (Human-in-the-loop) escalation
- LangGraph-based workflow

Out of Scope:

- Voice-based interaction (optional future)
- Multilingual translation (future)
- Real-time CRM integration

---

2. Architecture Diagram



---

3. Component Description

Document Loader

- Uses libraries like **PyPDFLoader / PDFPlumber**
- Handles:
  - Multi-page PDFs
  - Encoding issues
  - Metadata extraction
- Output: Clean raw text

---

a. Chunking Strategy

Why Chunking?

LLMs have token limits → large documents must be split.

Strategy:

- Chunk Size: **300–500 tokens**
- Overlap: **50–100 tokens**

Benefits:

- Maintains **context continuity**
- Improves **retrieval accuracy**
- Prevents **information loss**

---

b. Embedding Model

Sentence Transformer-based embedding model (e.g., `all-MiniLM-L6-v2`) is used to convert document chunks into vector representations.

---

c. Vector Store (ChromaDB)

Features:

- Persistent storage
- Fast similarity search
- Supports metadata filtering
- Lightweight and developer-friendly

Storage Format:

- (chunk_id, embedding_vector, metadata)

---

d. Retriever

Function:

- Accepts query embedding
- Finds **Top-K similar chunks**

Techniques:

- Cosine similarity
- Approximate nearest neighbor (ANN)

Output:

- Ranked list of relevant chunks

---

e. LLM (Language Model)

The system uses **Groq API** to run high-performance LLMs (e.g., LLaMA 3 / Mixtral) for generating responses.
Groq is chosen for:

- Extremely low latency
- High throughput
- Cost efficiency compared to traditional APIs

---

f. Graph Workflow Engine (LangGraph)

Purpose:

- Manage execution flow using **state machine**
- Enable:
    - Conditional routing
    - Multi-step reasoning
    - HITL integration

Advantage:

Better than linear pipelines → supports branching logic

---

g. Routing Layer

Decision Logic:

- If context sufficient → Answer
- If ambiguous → Clarification
- If low confidence → HITL

---

h. HITL Module (Human-in-the-Loop)

Function:

- Escalate complex queries
- Store unresolved queries
- Allow human response injection

Benefits:

- Improves reliability
- Handles edge cases

---

4. Data Flow

🔹 PDF → Answer Flow

PDF Upload
↓

Text Extraction
↓
Chunking
↓
Embedding Generation
↓
Storage in ChromaDB

🔹 Query Lifecycle

User Query
↓
Query Embedding
↓
Similarity Search (Retriever)
↓
Top-K Relevant Chunks
↓
LLM Processing
↓
LangGraph Decision
↓
Answer / Clarification / HITL
↓
Final Output

---

5. Technology Choices

🔹 ChromaDB

- Zero setup (local)
- Ideal for prototyping + production
- Supports metadata filtering
- Faster than traditional DBs for vector search

🔹 LangGraph

- Stateful orchestration
- Supports branching workflows

- Enables HITL integration easily
- Better control than LangChain chains

🔹 LLM Options

Model	Pros	Cons
Groq (LLaMA / Mixtral via Groq)	cost-effective, easy API	Limited model variety, depends on hosted models
LLaMA	Open-source	Setup complexity
Mistral	Fast & efficient	Slightly lower reasoning

🔹 Additional Tools

Tool	Purpose
FastAPI	Backend API
Streamlit	UI
LangChain	RAG utilities
PyPDF	PDF parsing

---

6. Scalability Considerations

🔹 Handling Large Documents

Challenges:

- Memory constraints
- Token limits

Solutions:

- Chunking + indexing

- Incremental ingestion
- Parallel processing

Increasing Query Load

Problems:

- High API latency
- LLM cost increase

Solutions:

- Response caching (Redis)
- Async API calls
- Load balancing
- Horizontal scaling

Latency Optimization

Techniques:

- Reduce chunk size
- Optimize Top-K (3–5)
- Use smaller embedding models
- Cache frequent queries
- Precompute embeddings

Storage Optimization

- Compress embeddings
- Use hybrid search (keyword + vector)
- Periodic cleanup of unused sessions

Reliability Improvements

- Fallback LLM

- Retry mechanism
- HITL backup